

PPL Lecture-4

Program Execution

and

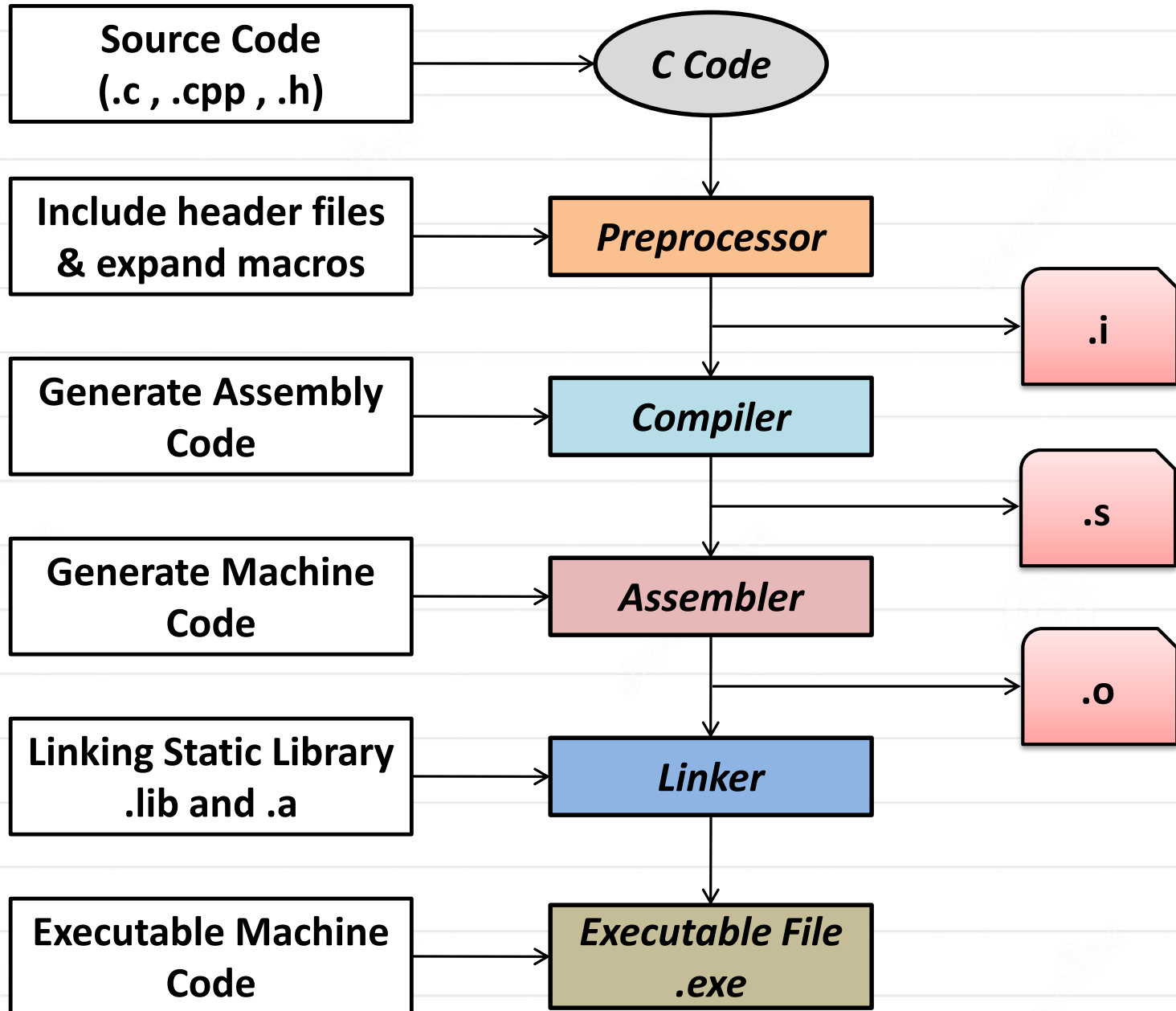
Concept of a Compiled Program

First C Program

```
#include<stdio.h>
int main()
{
    printf("Hello World");
    return 0;
}
```

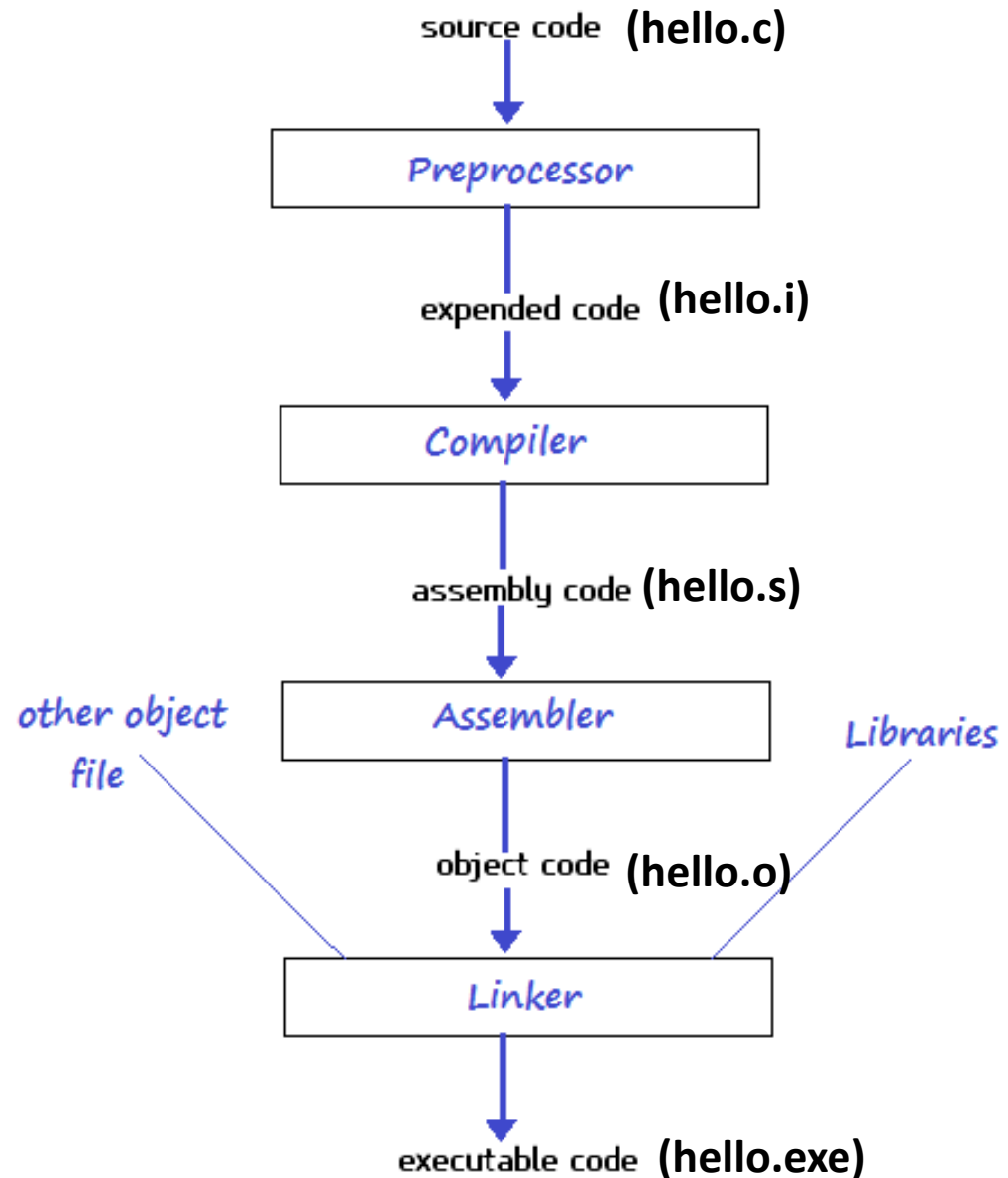
- This program prints Hello World on the screen when we execute it

Stages of Program Compilation



How does a C program executes?

- 1) C program (hello.c) is sent to preprocessor first. The preprocessor is responsible to convert preprocessor directives into their respective values. The preprocessor generates an expanded source code (hello.i)
- 2) Expanded source code is sent to compiler which compiles the code and converts it into assembly code (hello.s)
- 3) The assembly code is sent to assembler which assembles the code and converts it into object code. Now a simple.obj file is generated.
- 4) The object code is sent to linker which links it to the library such as header files. Then it is converted into executable code. A simple.exe file is generated.
- 5) The executable code is sent to loader which loads it into memory and then it is executed. After execution, output is sent to console.



Preprocessor

- The C preprocessor `cpp` expands all those macros definitions and include statements (and anything else that starts with a `#`) and passes the result to the actual compiler.
- Command: `$ gcc -E hello.c > hello.i`
 - `hello.c` is the source program, `hello.i` in the ASCII intermediate code

Compiler

- It takes output of the preprocessor and generates assembly source code
- Command: `$ gcc -S hello.i`
 - This will produce `hello.s`
- A compiler is more intelligent than an assembler. It checks all kinds of limits, ranges, errors etc.
- But its program run time is more and occupies a larger part of the memory.

Assembler

- An assembler translates assembly language programs into machine code.
- The output of an assembler is called an *object file*, which contains a combination of machine instructions as well as the data required to place these instructions in memory.
- Command: `$ gcc -c hello.s`
 - This will produce `hello.o`

Linker

- Linker is a computer program that links and merges various object files together in order to make an executable file.
- All these files might have been compiled by separate assemblers. The major task of a linker is to search and locate referenced module/routines in a program and to determine the memory location where these codes will be loaded, making the program instruction to have absolute references.
- Command: `$ gcc -O hello.o`
 - This will produce `a.out`

Loader

- Loader is a part of operating system and is responsible for loading executable files into memory and execute them.
- It calculates the size of a program (instructions and data) and creates memory space for it.
- It initializes various registers to initiate execution.

Object file and Executable file

- An **object file** is a file containing object code, meaning relocatable format machine code that is usually not directly executable.
- Object file has .o extension e.g. hello.o
- Object files are produced by an assembler, compiler, or other language translator, and used as input to the linker, which in turn typically generates an executable file.
- An **executable file** is formed by linking the Object files and has .exe extension.
- Object file contains low level instructions which can be understood by the CPU.
- That is why it is also called machine code.

Difference between Object file and Executable file

Both are binary files but the differences between those are:

- 1) we can execute an executable file while we cannot execute an object file.
- 2) An object file is a file where linker has not yet linked to the libraries, so you get an object file just before linking to the libraries, so still some of the symbols or function definitions are not yet resolved which are actually present in the libraries, and that's why we cannot execute it.

Difference between Interpreter and Compiler

Interpreter

1. Translates program one statement at a time
2. It takes less amount of time to analyze the source code but the overall execution time is slower
3. No intermediate object code is generated, hence are memory efficient
4. Continues translating the program until the first error is met, in which case it stops. Hence debugging is easy
5. Programming language like Python, Ruby use Interpreters

Compiler

- Scans the entire program and translates it as a whole into machine code
- It takes large amount of time to analyze the source code but the overall execution time is comparatively faster
- Generates intermediate object code which further requires linking, hence requires more memory
- It generates the error message only after scanning the whole program. Hence debugging is comparatively hard
- Programming language like C, C++ use compilers